Comparison Operators

Naturally, there will be many times when values need to be compared in order for your program to function correctly. We've already seen these in action (e.g., Chapter 4, "Logic and Control"), but let's quickly review comparison operators in the context of numbers and other numerical data:

```
$a = 1;
$b = 2;
$strA = '1';
$strB = '2';

var_dump($a == $b); // True if values are equal
var_dump($a == $strA); // True (string is cast to int)
var_dump($a === $strA); // True if values AND types are equal
var_dump($a != $b); // True if values are not equal
var_dump($a <> $b); // Same as !=
var_dump($a !== $strA); // True if values are not equal OR types are
                        different
var_dump($a < $b); // True if $a is less than $b
var_dump($a > $b); // True if $a is more than $b
var_dump($a <= $b); // True if $a is less than or equal to $b
var_dump($a >= $b); // True if $a is more than or equal to $b
var_dump($a <=> $b); // Returns -1, 0, or 1 if $a is less than, equal to,
                     or greater than $b
```

There are some potential pitfalls to be aware of when comparing numbers, especially when PHP's type coercion is triggered (e.g., when comparing an integer to a numerical string). Don't worry; we'll get to them soon.

Bitwise Operators

Binary operations are supported by PHP: operators exist to evaluate and manipulate data at a binary level. It should be noted that PHP is not an optimal language for bitwise operation due to limitations on how data is stored. Your code might only care about a small integer that can be represented as a single byte value (*255* or smaller), but PHP uses an internal C data structure called a `zend_long` to store it, and those are fixed at four or eight bytes in length depending on the platform.